BUG BOUNTY FIELD GUIDE

Netlas Dorking

Lucene Query Syntax, Real-World Dorks & Automation for Bug Bounty Hunters and Penetration Testers

- Domain-Centric Discovery
- 2025 Edition

EASM & INTERNET INTELLIGENCE

Table of Contents

1. Introduction to Netlas

Netlas is a relatively new (founded 2022) internet asset search engine built on Apache Lucene (Elasticsearch). Unlike legacy scanners, Netlas indexes **both IP addresses and domain names** simultaneously — maintaining a separate DNS index alongside its service scan data. This dual approach makes Netlas exceptionally powerful for **domain-centric reconnaissance** and virtual host discovery. 🌐

For bug bounty hunters, Netlas offers several unique advantages: it finds significantly more unique web resources than competitors due to virtual host handling (~344 million HTTP services vs ~145 million on Shodan), provides built-in DNS and WHOIS lookup tools, supports regex body searches, and offers CVE correlation out of the box. 🎯

> **Why Netlas Matters for Bug Bounty**
>
> **Domain + IP Indexing:** Discovers virtual hosts and aliases others miss. **2x+ Web Coverage:** ~344M HTTP services indexed. **Lucene Power:** Regex, fuzzy search, wildcards, ranges built-in. **Built-in DNS/WHOIS:** No need for external dig/whois tools. **CVE Native:** Direct vulnerability correlation with exploit flags. **Private Scanner:** On-demand subnet scanning in the familiar UI.

How Netlas Works

Netlas operates four core search tools:

- **Responses Search** — HTTP/HTTPS service data with full body, headers, title, favicon
- **DNS Search** — A, MX, TXT, NS, CNAME records and DNS content analysis
- **IP Search** — Host-level data with ports, banners, geolocation, ASN
- **Certificates Search** — Full SSL/TLS certificate chains with Subject, Issuer, SANs
- **WHOIS Search** — Domain registration data, contacts, registrars

> **Netlas Search Tools Workflow**
>
> **1. Responses Search** for web app recon (titles, bodies, headers). **2. DNS Search** for subdomain enumeration and record analysis. **3. IP Search** for network mapping and port discovery. **4. Certificates Search** for SSL-based asset correlation. **5. WHOIS Search** for org contact intelligence and domain history.

Account Tiers

Table 1 Netlas Pricing Plans

Plan	Cost	API Requests/Month	API Rate Limit
Community	Free	Limited	1 req/sec
Freelancer	$49/mo	30,000	1 req/sec
Business	$249/mo	Unlimited	2 req/sec
Corporate	$830/mo	Unlimited	5 req/sec
Enterprise	Custom	Unlimited	Higher

Netlas Access Notes

Netlas offers **API and CLI access even on the free Community tier** — a major advantage over competitors. The web interface provides query examples for each tool. Regex search, vulnerability search, and tag search require paid plans. Sign up at **netlas.io**.

2. Netlas Query Language

Netlas is built on Apache Lucene — the same engine powering Elasticsearch and Kibana. This means standard Lucene query syntax applies: field:value pairs, boolean logic, ranges, wildcards, fuzzy matching, and regular expressions. Mastering this syntax unlocks Netlas's full potential. 📝

2.1 Basic Syntax & Fields

SYNTAX

Table 2 Basic Query Syntax

Pattern	Description	Example
`field:value`	Exact match in field	`host:google.com`
`field:"value"`	Phrase match (multi-word)	`http.title:"Login Page"`
`field:*`	Field exists (any value)	`http.title:*`
`field.subfield`	Nested field (dot notation)	`geo.country:"United States"`
`keyword`	Search default fields	`netlas` (no field)

> **Important: No Spaces Around Colons**
>
> In Lucene syntax, there must be **no spaces** between the field name, colon, and value. Spaces act as condition separators. Wrong: `host : google.com`. Correct: `host:google.com`.

2.2 Logical & Range Operators

LOGIC

Table 3 Logical & Range Operators

Operator	Description	Example
`AND` / `&&`	Both conditions must match	`port:443 AND protocol:https`
`OR` / `||`	Either condition matches	`port:80 OR port:443`
`NOT` / `!`	Exclude condition	`protocol:ssh NOT port:22`
`()`	Grouping for precedence	`(port:8080 OR port:8088) AND protocol:http`
`[a TO b]`	Inclusive range	`ip:[173.194.222.0 TO 173.194.222.255]`
`{a TO b}`	Exclusive range	`port:{80 TO 443}`
`⩾ ⩽`	One-sided comparison	`port:⩽1000`

2.3 Wildcards, Fuzziness & Regex

ADVANCED

Table 4 Advanced Search Operators

Operator	Description	Example
`*`	Multi-character wildcard	`domain:*example.com`
`?`	Single-character wildcard	`host:test?.example.com`
`~`	Fuzzy search (edit distance)	`title:Josph~` (catches Joseph)

`~1 / ~2`	Fuzzy with specific distance	`title:admin~1`
`/regex/`	Regular expression	`http.body:/password.*123/`
`CIDR`	IP subnet search	`host:192.168.0.0/16`
`now-Xd`	Relative date range	`@timestamp:[now-7d TO now]`

Regex Body Search Power

Netlas supports **regular expressions in response body searches** — a feature many competitors lack. This lets you hunt for patterns like API keys, JWT tokens, or specific error messages with surgical precision. Example: `http.body:/AKIA[0-9A-Z]{16}/` finds AWS access keys in HTML.

3. Core Search Filters

3.1 Host & Network Filters

NETWORK

Table 5 Host & Network Fields

Filter	Description	Example
`ip`	IPv4/IPv6 address	`ip:8.8.8.8`
`host`	Hostname or IP with port	`host:example.com`
`domain`	Base domain	`domain:example.com`
`uri`	Full URI including path	`uri:*example.com/admin*`
`port`	Port number	`port:9200`
`protocol`	Protocol (http, https, ssh, etc.)	`protocol:https`
`geo.country`	Country name	`geo.country:"United States"`
`geo.city`	City name	`geo.city:London`
`asn`	Autonomous System Number	`asn:15169`

`path`	URL path	`path:/admin`

3.2 HTTP & Content Filters

WEB CONTENT

Table 6 HTTP & Content Fields

Filter	Description	Example
`http.title`	HTML page title	`http.title:"Login"`
`http.body`	Response body content	`http.body:admin`
`http.headers.server`	Server header	`http.headers.server:nginx`
`http.headers.www_authenticate`	Auth header	`http.headers.www_authenticate:Basic`
`http.status`	HTTP status code	`http.status:200`
`http.meta`	Meta tag content	`http.meta:generator`
`http.favicon`	Favicon hash (mmh3)	`http.favicon:-335242539`
`banner`	Service banner text	`banner:"OpenSSH"`

3.3 Certificate & Tag Filters

CERT & TAGS

Table 7 Certificate & Tag Fields

Filter	Description	Example
`certificate.subject.CN`	Cert Common Name	`certificate.subject.CN:"*.example.com"`
`certificate.subject.organization`	Org in cert subject	`certificate.subject.organization:"Google LLC"`

`certificate.issuer.organization`	Cert issuer org	`certificate.issuer.organization:"DigiCert"`
`certificate.issuer.email_address`	Issuer contact email	`certificate.issuer.email_address:*`
`certificate.fingerprint_sha256`	SHA256 fingerprint	`certificate.fingerprint_sha256:abc123...`
`tag.name`	Product tag name	`tag.name:"adobe_coldfusion"`
`tag.category`	Tag category	`tag.category:"cms"`

3.4 CVE & Vulnerability Filters

VULNERABILITIES

Table 8 CVE Fields

Filter	Description	Example
`cve`	Any CVE present	`cve:*`
`cve.name`	Specific CVE ID	`cve.name:CVE-2021-44228`
`cve.description`	CVE description text	`cve.description:log4j`
`cve.has_exploit`	Has public exploit	`cve.has_exploit:true`
`cve.cvss_score`	CVSS score range	`cve.cvss_score:[9.0 TO 10.0]`

> **Tag-Based Search**
>
> Netlas auto-tags services with product names. Tags cover Blogs (wordpress, medium), CDN (cloudflare, google_cloud), CMS (joomla, drupal), Ecommerce (magento, opencart), and many more. Click the tag icon in the search bar to browse all available tags. `tag.name` requires Business+ plan.

4. Bug Bounty Dorks by Category

These Netlas queries are crafted for real-world bug bounty hunting. Every dork targets a specific exposure type with copy-paste precision. 🎯

4.1 Admin Panels & Consoles

HIGH-IMPACT

ADMIN Jenkins Dashboard

Unprotected Jenkins instances expose build pipelines, secrets, and script console RCE.

```
http.title:"Dashboard [Jenkins]" AND http.headers.server:*
```

ADMIN Apache Tomcat Manager

Tomcat manager allows WAR deployment for remote code execution.

```
http.title:"Apache Tomcat" AND http.body:manager
```

ADMIN phpMyAdmin

phpMyAdmin interfaces with weak credentials = full database compromise.

```
http.title:"phpMyAdmin" AND http.body:pma_username
```

ADMIN Grafana Login

Grafana exposes metrics, logs, and potentially admin panels.

```
http.title:"Grafana" AND tag.name:grafana
```

ADMIN **Kubernetes Dashboard**

Exposed K8s dashboards allow pod execution and cluster takeover.

```
http.title:"Kubernetes Dashboard" AND http.body:Kubernetes
```

ADMIN **Docker Registry**

Unauthenticated registries leak container images and source artifacts.

```
http.title:"Docker Registry" AND port:5000
```

ADMIN **Kibana Elasticsearch**

Kibana exposes indexed logs and allows Elasticsearch query execution.

```
http.title:"Kibana" AND port:5601
```

ADMIN **Cisco ASDM / VPN**

Cisco security devices with known CVEs and default configurations.

```
http.title:"Cisco ASDM" AND certificate.subject.CN:*cisco*
```

4.2 Databases & Data Leaks

SENSITIVE DATA

DATABASE **Open Elasticsearch**

Unauthenticated Elasticsearch exposes all indexed data — P1 severity.

```
port:9200 AND http.body:cluster_name AND http.body:indices
```

DATABASE **Exposed MongoDB**

MongoDB without auth exposes entire databases on default port 27017.

```
port:27017 AND banner:MongoDB
```

DATABASE **Redis Unprotected**

Redis without AUTH exposes cached data and may allow config rewrite RCE.

```
port:6379 AND banner:redis_version
```

LEAKED **AWS S3 Buckets**

Misconfigured S3 with public ListBucketResult XML exposed.

```
http.body:ListBucketResult AND http.body:s3.amazonaws.com
```

LEAKED **Git Config Exposure**

.git/config files leak repository URLs, paths, and sometimes credentials.

```
http.body:"[core]" AND http.body:repositoryformatversion
```

LEAKED **Open Directory Listings**

Directory indexes expose backups, configs, and sensitive files.

```
http.title:"Index of /" AND http.body:"Parent Directory"
```

4.3 Cloud & DevOps Exposure

CLOUD INFRA

CLOUD

Swagger / OpenAPI Docs

Exposed Swagger UI reveals API endpoints, schemas, and test tokens.

```
http.title:"Swagger UI" AND http.body:swagger-ui
```

CLOUD

GraphQL Playground

GraphQL introspection allows full schema discovery and data extraction.

```
http.title:"GraphQL Playground" AND http.body:graphql
```

CLOUD

MinIO Storage Console

MinIO interfaces may expose buckets without authentication.

```
http.title:"MinIO Browser" AND http.body:MinIO
```

DEVOPS

Prometheus Metrics

Exposed /metrics endpoints leak system metrics and sometimes paths.

```
http.body:process_resident_memory_bytes OR http.body:http_requests_total
```

DEVOPS

Docker-Compose / .env Leaks

Config files in web roots leak credentials, API keys, and secrets.

```
http.body:DB_PASSWORD AND http.body:APP_KEY
```

CLOUD

Firebase / App Configs

Exposed mobile app configs and cloud function endpoints.

```
http.body:firebaseio.com OR http.body:appspot.com
```

4.4 API Keys & Secrets

SECRETS

SECRET AWS Access Keys

AWS access key IDs (AKIA/ASIA prefix) exposed in source code.

```
http.body:/AKIA[0-9A-Z]{16}/
```

SECRET Google API Keys

Google Maps, Firebase, and Cloud API keys in JavaScript.

```
http.body:/AIza[0-9A-Za-z_-]{35}/
```

SECRET Stripe Keys

Stripe publishable and secret keys leaked in frontend or configs.

```
http.body:pk_live_ OR http.body:sk_live_
```

SECRET Slack / Discord Webhooks

Webhook URLs in source code allowing unauthorized message injection.

```
http.body:hooks.slack.com OR http.body:discord.com/api/webhooks
```

SECRET JWT / Bearer Tokens

Hardcoded JWT secrets or bearer tokens in frontend JS.

```
http.body:/Bearer eyJ[0-9A-Za-z_-]*/
```

SECRET **Private Keys in HTML**

RSA/EC private keys accidentally exposed in HTTP responses.

```
http.body:"BEGIN RSA PRIVATE KEY" OR http.body:"BEGIN OPENSSH PRIVATE KEY"
```

4.5 Vulnerable Services & CVEs

CVE HUNTING

CVE **Log4j / Log4Shell**

Java apps with Log4j vulnerable to JNDI injection RCE.

```
cve.name:CVE-2021-44228 AND http.body:log4j
```

CVE **F5 BIG-IP (CVE-2022-1388)**

BIG-IP iControl REST auth bypass — critical unauthenticated RCE.

```
http.title:"BIG-IP" AND tag.name:f5_bigip
```

CVE **Apache Struts (CVE-2017-5638)**

Apache Struts with OGNL expression injection on legacy systems.

```
tag.name:apache_struts AND http.status:200
```

CVE **Exploitable CVEs with Public Exploits**

Find vulnerable services that have known public exploits ready to use.

```
cve.has_exploit:true AND cve.cvss_score:[9.0 TO 10.0]
```

CVE **Vulnerable WebLogic Servers**

Oracle WebLogic with known CVEs and remote code execution history.

```
cve.description:weblogic AND cve.has_exploit:true
```

CVE **SQL Injection Error Pages**

Pages showing MySQL/PostgreSQL error messages — potential SQLi vectors.

```
http.body:mysql_fetch_array AND http.body:warning
```

5. Targeting Specific Organizations

Netlas's domain-centric approach makes it exceptional for organization-targeted reconnaissance. The dual IP+domain index reveals virtual hosts and aliases that pure IP scanners miss. 🗺️

Organization Enumeration Strategy

STEP-BY-STEP TARGETING

```
# Step 1: Base domain wildcard search
domain:*.example.com

# Step 2: Find all subdomains via DNS
dns.name:*.example.com

# Step 3: Certificate-based discovery
certificate.subject.CN:*.example.com

# Step 4: Combine domain + specific service
domain:*.example.com AND (http.title:admin OR http.title:login)

# Step 5: Exclude cloud providers to find bare metal
domain:*.example.com NOT domain:*.amazonaws.com NOT domain:*.cloudflare.com

# Step 6: Find non-standard ports
domain:*.example.com AND NOT port:(80 OR 443 OR 8080 OR 8443)

# Step 7: HTTPS on unusual ports (often dev/staging)
domain:*.example.com AND protocol:https AND NOT port:443
```

DNS-Based Subdomain Enumeration

> **DNS Search Tool Power**
>
> Netlas's separate **DNS Search** tool indexes A, MX, TXT, NS, and CNAME records. Search by DNS content to find SPF records pointing to third-party services (revealing email infrastructure), MX records (email providers), or TXT records with verification tokens.

DNS RECON QUERIES

```
# Find all subdomains via DNS A records
dns.name:*.example.com

# Discover email infrastructure via MX records
dns.mx:*example.com

# SPF records revealing third-party services
dns.txt:mailgun AND domain:*example.com

# Find domains with DMARC records (email security posture)
dns.txt:"v=DMARC1" AND domain:*example.com

# CNAME-based discovery (often reveals CDNs and PaaS)
dns.cname:*.herokudns.com AND domain:*example.com
```

Certificate SAN Enumeration

CERTIFICATE RECON

```
# Find all SANs for a target domain
certificate.subject.CN:*.example.com

# Find related domains sharing the same issuer
certificate.issuer.organization:"DigiCert" AND certificate.subject.CN:*example*

# Expired certificates (may indicate abandoned infrastructure)
certificate.subject.CN:*.example.com AND NOT certificate.is_valid:true

# Wildcard certificate enumeration
certificate.subject.CN:"*.example.com"
```

WHOIS Contact Intelligence

WHOIS RECON

```
# Find all domains registered by the same org
whois.organization:"Example Company"

# Find domains by registrant email (correlates acquisitions)
whois.email:*@example.com

# Discover related domains via registrar patterns
whois.registrar:"GoDaddy" AND whois.organization:"Example"
```

Favicon Hash Matching

FAVICON HASH WORKFLOW

```
# 1. Get favicon from target
curl -s https://target.com/favicon.ico | python3 -c "
import sys, mmh3, base64
img = sys.stdin.buffer.read()
print(mmh3.hash(base64.encodebytes(img)))
"

# 2. Search Netlas by hash
http.favicon:-335242539

# 3. Combine with domain for precision
http.favicon:-335242539 AND domain:*.target.com

# 4. Find global shadow deployments
http.favicon:-335242539 AND NOT domain:*.target.com
```

6. Netlas API & Automation

Netlas provides a Python library, CLI tool, and REST API — all accessible even on the free tier. This makes automation easy and cost-effective for bug bounty hunters. 🤖

Python SDK Setup

INSTALLATION & AUTH

```
# Install Netlas Python library
pip install netlas

# Get API key from https://app.netlas.io/profile/
# Set as environment variable or pass directly
export NETLAS_API_KEY=your_api_key
```

Python API Examples

PYTHON: TARGETED ENUMERATION

```
import netlas

apikey = "YOUR_API_KEY"

# Create connection
netlas_connection = netlas.Netlas(api_key=apikey)

# Search responses for target admin panels
query = 'domain:*.example.com AND (http.title:"admin" OR http.title:"login")'
resp = netlas_connection.query(query=query)

for item in resp['items']:
    data = item['data']
      print(f"{data.get('ip', 'N/A')} | {data.get('uri', 'N/A')} | {data.get('http',
{}).get('title', 'N/A')}")
```

PYTHON: CERTIFICATE SUBDOMAIN DISCOVERY

```
import netlas

apikey = "YOUR_API_KEY"
netlas_connection = netlas.Netlas(api_key=apikey)

# Use certificates search tool
query = 'certificate.subject.CN:*.example.com'
resp = netlas_connection.query(query=query, datatype='certificates')

domains = set()
for item in resp['items']:
    cert = item['data'].get('certificate', {})
    subject = cert.get('subject', {})
    cn = subject.get('CN', '')
    if cn:
        domains.add(cn)
    # Also extract SANs
    for san in cert.get('subject_alt_names', []):
        domains.add(san)

for d in sorted(domains):
    print(d)
```

Netlas CLI

CLI COMMANDS

```
# Install CLI
pip install netlas

# Configure API key
netlas save_key YOUR_API_KEY

# Search responses
netlas search 'domain:*.example.com AND http.title:"Login"' -f json

# Search certificates
netlas search 'certificate.subject.CN:*.example.com' -f json

# Search DNS records
netlas search 'dns.name:*.example.com' -f json

# Search with specific fields output
netlas search 'port:9200 AND http.body:cluster_name' -f json | jq '.items[].data.uri'

# Download results to file
netlas search 'cve.has_exploit:true' -f json > exploits.json
```

cURL API One-Liners

CURL WORKFLOWS

```
# Basic responses search
curl             -X             GET             'https://app.netlas.io/api/responses/?
q=domain%3Aexample.com&source_type=include&start=0&fields=*' \
  -H 'accept: application/json' \
  -H 'X-API-Key: YOUR_API_KEY' | jq .

# Extract domains from results
curl          -s          -X          GET          'https://app.netlas.io/api/responses/?
q=domain%3A*.example.com&fields=domain,ip,port,http.title' \
  -H 'X-API-Key: YOUR_API_KEY' | jq -r '.items[].data | "\(.domain) | \(.ip):\(.port) |
\(.http.title)"'

# Certificate search for SAN enumeration
curl         -s         -X         GET         'https://app.netlas.io/api/certificates/?
q=certificate.subject.CN%3A*.example.com' \
  -H 'X-API-Key: YOUR_API_KEY' | jq -r '.items[].data.certificate.subject.CN'
```

7. Netlas vs Shodan vs Censys

Each search engine has unique strengths. Understanding these differences helps you choose the right tool for each reconnaissance phase. ⚖️

Table 9 Platform Comparison

Dimension	Netlas	Shodan	Censys
Founded	2022	2009	2015
Query Language	Lucene (Elasticsearch)	Simple filters	CenQL (structured)
Domain Indexing	Native + virtual hosts	Limited	Via certificates
HTTP Services	~344 million	~145 million	~51 million IPs
Regex Search	Full regex in body	Limited	Limited
Built-in DNS/WHOIS	Yes	No	Basic WHOIS
CVE Correlation	Native + exploit flag	Membership only	Via integrations
Private Scanner	Yes (Freelancer+)	No	ASM only
Free API	Yes (limited)	100 credits/mo	250 queries/mo
Cheapest Paid	$49/mo (30K requests)	$49/mo	$69/mo (500 req)

When to Use Which

Table 10 Tool Selection Guide

Task	Best Tool	Why
Domain + virtual host discovery	Netlas	Native domain indexing, most web resources
Regex pattern in response body	Netlas	Full Lucene regex support
Built-in DNS/WHOIS lookup	Netlas	Integrated tools, no external utilities
CVE with exploit flag hunting	Netlas	Native cve.has_exploit filter
IoT / webcam discovery	Shodan	Largest IoT device database
Screenshots of web UIs	Shodan	Built-in screenshot capture
Certificate SAN enumeration	Censys	Deepest certificate chain indexing
SSL org field company mapping	Censys	O= field search is unmatched

High-accuracy service data	Censys	92% accuracy, fewer stale results
Quick budget recon	Netlas	Free API + most requests per dollar

Pro Strategy: The Trifecta

For maximum coverage, run the same target query on **all three** platforms. Netlas finds virtual hosts and domains others miss. Shodan surfaces IoT and screenshots. Censys provides the deepest certificate and accuracy validation. Cross-reference results for the most complete attack surface map.

8. Integration & Workflow

Netlas integrates seamlessly into modern bug bounty reconnaissance pipelines. Here are proven workflows. 🔗

Netlas + Subfinder + httpx Pipeline

FULL RECON CHAIN

```
# Phase 1: Netlas domain discovery
curl          -s          -X          GET          'https://app.netlas.io/api/responses/?
q=domain%3A*.target.com&fields=domain' \
  -H 'X-API-Key: KEY' | jq -r '.items[].data.domain' | sort -u | tee netlas_domains.txt

# Phase 2: Expand with Subfinder
subfinder -d target.com -all -silent | anew netlas_domains.txt

# Phase 3: Resolve and validate
cat netlas_domains.txt | dnsx -a -silent | tee resolved.txt

# Phase 4: Live HTTP probing with metadata
cat netlas_domains.txt | httpx -title -status-code -tech-detect -o live_http.txt

# Phase 5: Netlas deep scan on interesting hosts
for domain in $(cat netlas_domains.txt | head -50); do
                              curl       -s       "https://app.netlas.io/api/responses/?
q=domain:${domain}&fields=port,http.title,banner" \
      -H 'X-API-Key: KEY' >> netlas_deep.jsonl
done
```

Netlas + Nuclei CVE Validation

CVE HUNT CHAIN

```
# Step 1: Netlas query for exploitable CVEs
curl          -s          -X          GET          'https://app.netlas.io/api/responses/?
q=cve.has_exploit%3Atrue%20AND%20domain%3A*.target.com&fields=ip,port,uri,http.title' \
    -H  'X-API-Key:  KEY'  |  jq  -r  '.items[].data  |  "\(.uri)"'  |  sort  -u  |  tee
cve_targets.txt

# Step 2: Validate live hosts
cat cve_targets.txt | httprobe | tee live_cve.txt

# Step 3: Run Nuclei on confirmed targets
cat live_cve.txt | nuclei -t cves/ -severity critical,high -o nuclei_results.txt
```

Continuous Monitoring

MONITOR SCRIPT

```
#!/usr/bin/env python3
# netlas_monitor.py — Daily asset change detection

import netlas, json, datetime, os

apikey = "YOUR_API_KEY"
TARGET = "*.target.com"
STATE_FILE = "netlas_state.json"

conn = netlas.Netlas(api_key=apikey)

# Load known state
try:
    with open(STATE_FILE) as f:
        known = set(json.load(f))
except:
    known = set()

# Query current assets
query = f'domain:{TARGET}'
resp = conn.query(query=query, fields='ip,port,domain')

current = set()
for item in resp.get('items', []):
    d = item.get('data', {})
    current.add(f"{d.get('domain')}:{d.get('ip')}:{d.get('port')}")

# Detect new assets
new = current - known
if new:
    print(f"[{datetime.datetime.now()}] NEW ASSETS: {len(new)}")
    for asset in new:
        print(f"  - {asset}")
    # Notify or trigger further scans here

# Save updated state
with open(STATE_FILE, 'w') as f:
    json.dump(list(current), f)
```

9. Best Practices & Legal

Validation Checklist

Table 11 Before Reporting

Step	Action	Why

1	Confirm ownership via domain, cert subject, or WHOIS org	Shared hosting/CDNs create false positives
2	Verify exposure is in program scope	Out-of-scope reports waste triager time
3	Check if already reported	Duplicates don't earn rewards
4	Validate exploitability with manual testing	Confirmed = higher severity/payout
5	Document exact Netlas query + tool used	Shows reproducible methodology
6	Include timestamp and result screenshot	Proves finding is current

Report Template

Netlas Finding Report Template

Title: [P1] Unauthenticated Jenkins Instance at jenkins.target.com

Description: During passive reconnaissance using Netlas, I discovered an exposed Jenkins CI/CD instance belonging to [Target].

Netlas Query: `http.title:"Dashboard [Jenkins]" AND domain:*.target.com`

Asset: jenkins.target.com (IP 203.0.113.42) — confirmed live on [Date]

Impact: Unauthenticated Jenkins allows viewing build logs (potential secret exposure), running arbitrary builds, and potentially accessing the Script Console for remote code execution.

Remediation: Enable Jenkins security realm authentication. Restrict to VPN/internal network. Disable anonymous access. Review build logs for exposed secrets.

Legal & Ethical Reminders

Do not exploit without written permission. Netlas is passive, but accessing discovered systems without authorization violates CFAA, Computer Misuse Act, and similar laws. **Only report to programs where you have explicit scope. Never modify data, upload files, or execute commands on target systems. Follow responsible disclosure timelines.**

Recommended Resources

- **Netlas Search** — netlas.io (Official platform)
- **Netlas Docs** — docs.netlas.io (Query language & API reference)
- **Netlas Cookbook** — netlas.io/resources/cookbook (Query examples)

- **Netlas Python Library** — pip install netlas
- **Netlas Dorks Repo** — github.com/NetlasIO (Featured queries)
- **Nuclei Templates** — github.com/projectdiscovery/nuclei-templates